

Input,Reward

"This is a sentence.",0.8

"Another sentence.",0.2

...

在这个示例中，输入数据是一个句子，**奖励数据是一个实数值**，表示对输入数据的评价。每一行代表一个样本，第一列是输入数据，第二列是对应的奖励数据。

需要注意的是，具体的数据集格式可能会因任务类型、数据来源和使用的深度学习框架而有所不同。因此，在使用RM进行大语言模型训练时，建议根据具体任务和框架的要求来定义和处理数据集格式。

3.3 PPO（强化学习）的数据格式？

在大语言模型训练中，PPO（Proximal Policy Optimization，近端策略优化）是一种常用的强化学习算法。PPO的数据格式可以采用以下方式：

- 1) **输入数据**：输入数据是一个文本序列，通常是一个句子或者一个段落。每个样本可以是一个字符串或者是一个tokenized的文本序列。
- 2) **奖励数据**：奖励数据是与输入数据对应的奖励或评分。奖励可以是一个实数值，表示对输入数据的评价。也可以是一个离散的标签，表示对输入数据的分类。奖励数据可以是人工标注的，也可以是通过其他方式（如人工评估、模型评估等）得到的。
- 3) **动作数据**：动作数据是模型在给定输入数据下的输出动作。对于语言模型，动作通常是生成的文本序列。动作数据可以是一个字符串或者是一个tokenized的文本序列。
- 4) **状态数据**：状态数据是模型在给定输入数据和动作数据下的状态信息。对于语言模型，状态数据可以是模型的隐藏状态或其他中间表示。状态数据的具体形式可以根据具体任务和模型结构进行定义。
- 5) **数据集格式**：数据集可以以文本文件（如CSV、JSON等）或数据库的形式存储。每个样本包含输入数据、奖励数据、动作数据和状态数据。可以使用表格形式存储数据，每一列代表一个特征或标签。

下面是一个示例数据集的格式：

Input,Reward,Action,State

"This is a sentence.",0.8,"This is a generated sentence.",[0.1, 0.2, 0.3, ...]

"Another sentence.",0.2,"Another generated sentence.",[0.4, 0.5, 0.6, ...]

...

在这个示例中，输入数据是一个句子，奖励数据是一个实数值，动作数据是生成的句子，状态数据是模型的隐藏状态。每一行代表一个样本，第一列是输入数据，第二列是对应的奖励数据，第三列是生成的动作数据，第四列是状态数据。

需要注意的是，具体的数据集格式可能会因任务类型、数据来源和使用的深度学习框架而有所不同。因此，在使用PPO进行大语言模型训练时，建议根据具体任务和框架的要求来定义和处理数据集格式。

4. 有监督微调

4.1 微调方法是啥？如何微调？

微调 (Fine-tuning) 是一种迁移学习的方法，用于在一个预训练模型的基础上，通过在特定任务的数据上进行有监督训练，来适应该任务的要求并提高模型性能。微调利用了预训练模型在大规模通用数据上学习到的语言知识和表示能力，将其迁移到特定任务上。

下面是一般的微调步骤：

- 1) **预训练模型选择**：选择一个在大规模数据上进行预训练的模型作为基础模型。例如，可以选择一种预训练的语言模型，如BERT、GPT等。
- 2) **数据准备**：准备用于微调的特定任务数据集。这些数据集应包含任务相关的样本和相应的标签或目标。确保数据集与任务的特定领域或问题相关。
- 3) **构建任务特定的模型头**：根据任务的要求，构建一个特定的模型头 (task-specific head)。模型头是添加到预训练模型之上的额外层或结构，用于根据任务要求进行输出预测或分类。例如，对于文本分类任务，可以添加一个全连接层和softmax激活函数。
- 4) **参数初始化**：将预训练模型的参数作为初始参数加载到微调模型中。这些参数可以被视为模型已经学习到的通用语言表示。
- 5) **微调训练**：使用特定任务的数据集对模型进行有监督训练。这包括将任务数据输入到模型中，计算损失函数，并通过反向传播和优化算法（如梯度下降）更新模型参数。在微调过程中，只有模型头的参数会被更新，而预训练模型的参数会保持不变。
- 6) **调整超参数**：微调过程中，可以根据需要调整学习率、批量大小、训练迭代次数等超参数，以达到更好的性能。
- 7) **评估和验证**：在微调完成后，使用验证集或测试集对微调模型进行评估，以评估其在特定任务上的性能。可以使用各种指标，如准确率、精确率、召回率等。
- 8) **可选的后续微调**：根据实际情况，可以选择在特定任务的数据上进行进一步的微调迭代，以进一步提高模型性能。

微调的关键是在预训练模型的基础上进行训练，从而将模型的知识迁移到特定任务上。通过这种方式，可以在较少的数据和计算资源下，快速构建和训练高性能的模型。

4.2 微调和参数高效微调之间的区别是什么？

微调和参数高效微调是机器学习中用于提高预训练模型在特定任务上的性能的两种方法。

微调就是把一个预先训练好的模型用新的数据在一个新的任务上进一步训练它。整个预训练模型通常在微调中进行训练，包括它的所有层和参数。这个过程在计算上非常昂贵且耗时，特别是对于大型模型。

另一方面，**参数高效微调**是一种专注于只训练预训练模型参数的子集的微调方法。这种方法包括为新任务识别最重要的参数，并且只在训练期间更新这些参数。这样，PEFT可以显著减少微调所需的计算量。

4.3 PEFT 有什么优点？

在这里，只讨论PEFT相对于传统微调的好处。因此，理解为什么参数有效的微调比微调更有益。

- 1) **减少计算和存储成本**：PEFT只涉及微调少量额外的模型参数，而冻结预训练llm的大部分参数，从而显著降低计算和存储成本
- 2) **克服灾难性遗忘**：在LLM的全面微调期间，灾难性遗忘可能发生在模型忘记它在预训练期间学到的知识的地方。PEFT通过只更新几个参数来克服这个问题。
- 3) **低数据环境下更好的性能**：PEFT方法在低数据环境下的表现优于完全微调，并且可以更好地推广到域外场景。
- 4) **可移植性**：与全面微调的大检查点相比，PEFT方法使用户能够获得价值几mb的小检查点。这使得来自PEFT方法的训练权重易于部署和用于多个任务，而无需替换整个模型。
- 5) **与完全微调相当的性能**：PEFT仅使用少量可训练参数即可实现与完全微调相当的性能。

4.4 多种不同的高效微调方法对比

参数有效策略可能涉及多种技术：

- 1) **选择性层调整 (Selective Layer Tuning)**：可以只微调层的一个子集，而不是微调模型的所有层。这减少了需要更新的参数数量。
- 2) **适配器 (Adapters)**：适配器层是插入预训练模型层之间的小型神经网络。在微调过程中，只训练这些适配器层，保持预先训练的参数冻结。通过这种方式，适配器学习将预先训练的模型提取的特征适应新任务。
- 3) **稀疏微调 (Sparse Fine-Tuning)**：传统的微调会略微调整所有参数，但稀疏微调只涉及更改模型参数的一个子集。这通常是基于一些标准来完成的，这些标准标识了与新任务最相关的参数。
- 4) **低秩近似 (Low-Rank Approximations)**：另一种策略是用一个参数较少但在任务中表现相似的模型来近似微调后的模型。
- 5) **正则化技术 (Regularization Techniques)**：可以将正则化项添加到损失函数中，以阻止参数发生较大变化，从而以更“参数高效”的方式有效地微调模型。
- 6) **任务特定的头 (Task-specific Heads)**：有时，在预先训练的模型架构中添加一个任务特定的层或“头”，只对这个头进行微调，从而减少需要学习的参数数量。

4.5 当前高效微调技术存在的一些问题

当前的高效微调技术很难在类似方法之间进行直接比较并评估它们的真实性能，主要的原因如下所示：

- 1) **参数计算口径不一致**：参数计算可以分为三类：可训练参数的数量、微调模型与原始模型相比改变的参数的数量、微调模型和原始模型之间差异的等级。例如，DiffPruning更新0.5%的参数，但是实际参与训练的参数量是200%。这为比较带来了困难。尽管可训练的参数量是最可靠的存储高效指标，但是也不完美。Ladder-side Tuning使用一个单独的小网络，参数量高于LoRA或BitFit，但是因为反向传播不经过主网络，其消耗的内存反而更小。
- 2) **缺乏模型大小的考虑**：已有工作表明，大模型在微调中需要更新的参数量更小（无论是以百分比相对而言还是以绝对数量而论），因此（基）模型大小在比较不同PEFT方法时也要考虑到。
- 3) **缺乏测量基准和评价标准**：不同方法所使用的模型/数据集组合都不一样，评价指标也不一样，难以得到有意义的结论。
- 4) **代码实现可读性差**：很多开源代码都是简单拷贝Transformer代码库，然后进行小修小补。这些拷贝也不使用git fork，难以找出改了哪里。即便是能找到，可复用性也比较差（通常指定某个Transformer版本，没有说明如何脱离已有代码库复用这些方法）。

5. 推理

5.1 为什么大模型推理时显存涨的那么多还一直占着？

大语言模型进行推理时，显存涨得很多且一直占着显存不释放的原因主要有以下几点：

模型参数占用显存：大语言模型通常具有巨大的参数量，这些参数需要存储在显存中以供推理使用。因此，在推理过程中，模型参数会占用相当大的显存空间。

输入数据占用显存：进行推理时，需要将输入数据加载到显存中。对于大语言模型而言，输入数据通常也会占用较大的显存空间，尤其是对于较长的文本输入。

中间计算结果占用显存：在推理过程中，模型会进行一系列的计算操作，生成中间结果。这些中间结果也需要存储在显存中，以便后续计算使用。对于大语言模型而言，中间计算结果可能会占用较多的显存空间。

内存管理策略：某些深度学习框架在推理时采用了一种延迟释放显存的策略，即显存不会立即释放，而是保留一段时间以备后续使用。这种策略可以减少显存的分配和释放频率，提高推理效率，但也会导致显存一直占用的现象。

需要注意的是，显存的占用情况可能会受到硬件设备、深度学习框架和模型实现的影响。不同的环境和设置可能会导致显存占用的差异。如果显存占用过多导致资源不足或性能下降，可以考虑调整模型的批量大小、优化显存分配策略或使用更高性能的硬件设备来解决问题。

5.2 大模型在GPU和CPU上推理速度如何

大语言模型在GPU和CPU上进行推理的速度存在显著差异。一般情况下，GPU在进行深度学习推理任务时具有更高的计算性能，因此大语言模型在GPU上的推理速度通常会比在CPU上更快。

以下是GPU和CPU在大语言模型推理速度方面的一些特点：

- 1) **GPU推理速度快**：GPU具有大量的并行计算单元，可以同时处理多个计算任务。对于大语言模型而言，GPU可以更高效地执行矩阵运算和神经网络计算，从而加速推理过程。
- 2) **CPU推理速度相对较慢**：相较于GPU，CPU的计算能力较弱，主要用于通用计算任务。虽然CPU也可以执行大语言模型的推理任务，但由于计算能力有限，推理速度通常会较慢。
- 3) **使用GPU加速推理**：为了充分利用GPU的计算能力，通常会使用深度学习框架提供的GPU加速功能，如CUDA或OpenCL。这些加速库可以将计算任务分配给GPU并利用其并行计算能力，从而加快大语言模型的推理速度。

需要注意的是，推理速度还受到模型大小、输入数据大小、计算操作的复杂度以及硬件设备的性能等因素的影响。因此，具体的推理速度会因具体情况而异。一般来说，使用GPU进行大语言模型的推理可以获得更快的速度。

5.3 推理速度上，INT8和FP16比起来怎么样？

在大语言模型的推理速度上，使用INT8（8位整数量化）和FP16（半精度浮点数）相对于FP32（单精度浮点数）可以带来一定的加速效果。这是因为INT8和FP16的数据类型在表示数据时所需的内存和计算资源较少，从而可以加快推理速度。

具体来说，INT8在相同的内存空间下可以存储更多的数据，从而可以在相同的计算资源下进行更多的并行计算。这可以提高每秒推理操作数（Operations Per Second, OPS）的数量，加速推理速度。

FP16在相对较小的数据范围内进行计算，因此在相同的计算资源下可以执行更多的计算操作。虽然FP16的精度相对较低，但对于某些应用场景，如图像处理和语音识别等，FP16的精度已经足够满足需求。

需要注意的是，INT8和FP16的加速效果可能会受到硬件设备的支持程度和具体实现的影响。某些硬件设备可能对INT8和FP16有更好的优化支持，从而进一步提高推理速度。

综上所述，使用INT8和FP16数据类型可以在大语言模型的推理过程中提高推理速度，但需要根据具体场景和硬件设备的支持情况进行评估和选择。

5.4 大模型有推理能力吗？

逻辑推理是大语言模型“智能涌现”出的核心能力之一，好像AI有了人的意识一样。而推理能力的关键，在于一个技术——思维链（Chain of Thought, CoT）。当模型规模足够大的时候，LLM本身是具备推理能力的。

在简单推理问题上，LLM已经达到了很好的能力；复杂推理问题上，还需要更多深入的研究。

5.5 大模型生成时的参数怎么设置？

- **Temperature**：用于调整随机从生成模型中抽样的程度，使得相同的提示可能会产生不同的输出。温度为 0 将始终产生相同的输出，该参数设置越高随机性越大。
- **波束搜索宽度**：波束搜索是许多 NLP 和语音识别模型中常用的一种算法，作为在给定可能选项的情况下选择最佳输出的最终决策步骤。波束搜索宽度是一个参数，用于确定算法在搜索的每个步骤中应该考虑的候选数量。
- **Top p**：动态设置tokens候选列表的大小^{**}。^{**} 将可能性之和不超过特定值的top tokens列入候选名单。Top p 通常设置为较高的值（如 0.75），目的是限制可能被采样的低概率 token 的长度。
- **Top k**：允许其他高分tokens有机会被选中^{**}。^{**} 这种采样引入的随机性有助于在很多情况下生成的质量。Top k 参数设置为 3 则意味着选择前三个tokens。

若 Top k 和 Top p 都启用，则 Top p 在 Top k 之后起作用

5.6 有哪些省内存的大语言模型训练/微调/推理方法？

有一些方法可以帮助省内存的大语言模型训练、微调和推理，以下是一些常见的方法：

- 1) **参数共享 (Parameter Sharing)**：通过共享模型中的参数，可以减少内存占用。例如，可以在不同的位置共享相同的嵌入层或注意力机制。
- 2) **梯度累积 (Gradient Accumulation)**：在训练过程中，将多个小批次的梯度累积起来，然后进行一次参数更新。这样可以减少每个小批次的内存需求，特别适用于GPU内存较小的情况。
- 3) **梯度裁剪 (Gradient Clipping)**：通过限制梯度的大小，可以避免梯度爆炸的问题，从而减少内存使用。
- 4) **分布式训练 (Distributed Training)**：将训练过程分布到多台机器或多个设备上，可以减少单个设备的内存占用。分布式训练还可以加速训练过程。
- 5) **量化 (Quantization)**：将模型参数从高精度表示（如FP32）转换为低精度表示（如INT8或FP16），可以减少内存占用。量化方法可以通过减少参数位数或使用整数表示来实现。
- 6) **剪枝 (Pruning)**：通过去除冗余或不重要的模型参数，可以减少模型的内存占用。剪枝方法可以根据参数的重要性进行选择，从而保持模型性能的同时减少内存需求。
- 7) **蒸馏 (Knowledge Distillation)**：使用较小的模型（教师模型）来指导训练较大的模型（学生模型），可以从教师模型中提取知识，减少内存占用。
- 8) **分块处理 (Chunking)**：将输入数据或模型分成较小的块进行处理，可以减少内存需求。例如，在推

理过程中，可以将较长的输入序列分成多个较短的子序列进行处理。

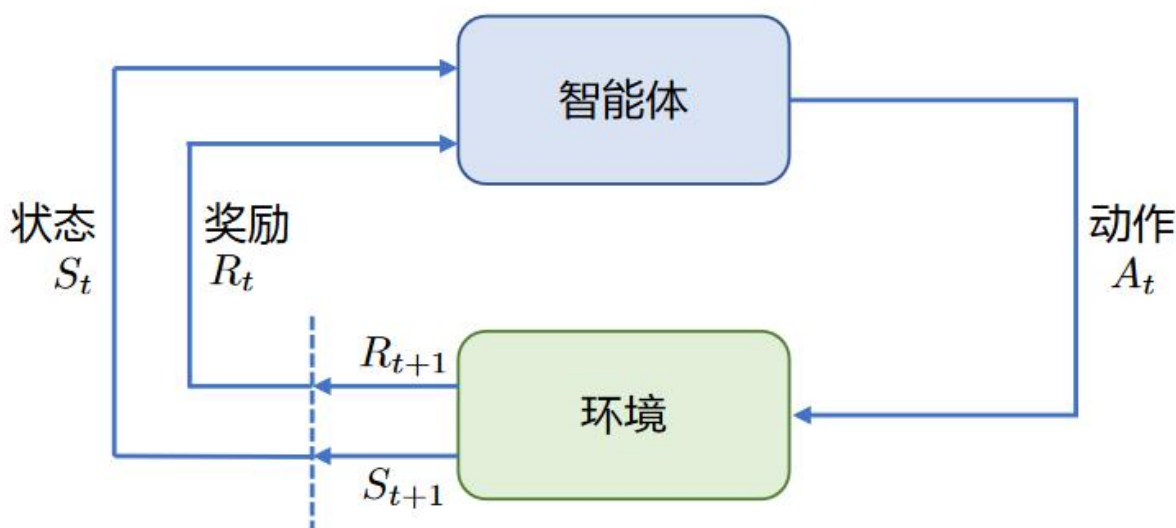
这些方法可以结合使用，根据具体场景和需求进行选择 and 调整。同时，不同的方法可能对不同的模型和任务有不同的效果，因此需要进行实验和评估。

6. 强化学习

6.1 简单介绍强化学习？

强化学习 (Reinforcement Learning, RL) 研究的问题是智能体 (Agent) 与环境 (Environment) 交互的问题，其目标是使智能体在复杂且不确定的环境中最大化奖励 (Reward)。

强化学习基本框架如图所示，主要由两部分组成：**智能体**和**环境**。在强化学习过程中，智能体与环境不断交互。智能体在环境中获取某个状态后，会根据该状态输出一个动作 (Action)，也称为决策 (Decision)。动作会在环境中执行，环境会根据智能体采取的动作，给出下一个状态以及当前动作所带来的奖励。智能体的目标就是尽可能多地从环境中获取奖励。本节中将介绍强化学习的基本概念、强化学习与有监督学习的区别，以及在大语言模型中基于人类反馈的强化学习流程。



强化学习在大语言模型上的重要作用可以概括为以下几个方面：

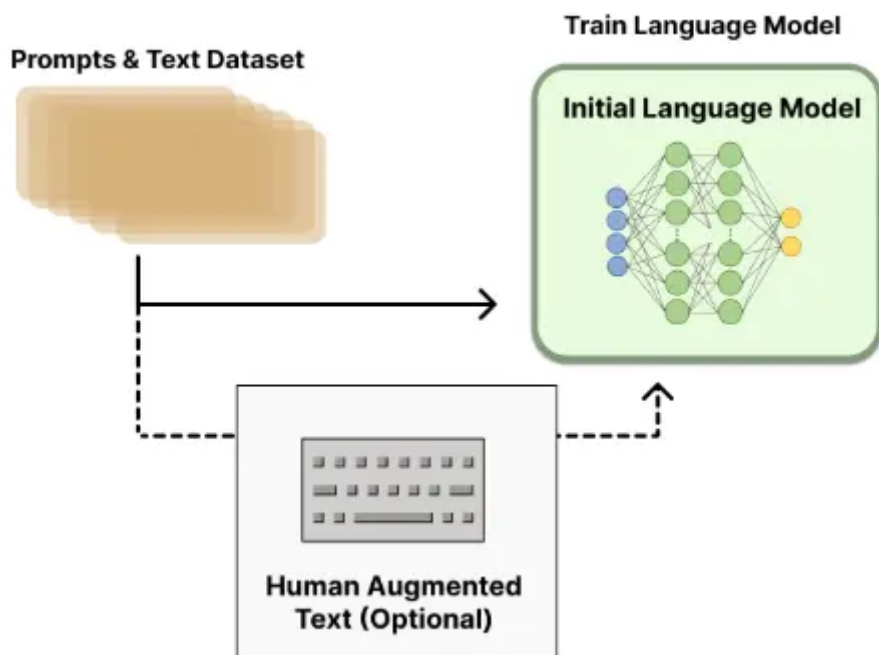
- 1) **强化学习比有监督学习更可以考虑整体影响**：有监督学习针对单个词元进行反馈，其目标是要求模型针对给定的输入给出的确切答案。而强化学习是针对整个输出文本进行反馈，并不针对特定的词元。
- 2) **强化学习更容易解决幻觉问题**：有监督学习算法非常容易使得求知型查询产生幻觉。在模型并不包含或者知道答案的情况下，有监督训练仍然会促使模型给出答案。而使用强化学习方法，则可以通过定制奖励函数，将正确答案赋予非常高的分数，放弃回答的答案赋予中低分数，不正确的答案赋予非常高的负分，使得模型学会依赖内部知识选择放弃回答，从而在一定程度上缓解模型幻觉问题。
- 3) **强化学习可以更好的解决多轮对话奖励累积问题**：使用强化学习方法，可以通过构建奖励函数，将当

前输出考虑整个对话的 背景和连贯性

6.2 简单介绍一下 RLHF?

RLHF就是基于人类反馈（Human Feedback）对语言模型进行强化学习（Reinforcement Learning），一般分为以下三个步骤：

- 1) 预训练语言模型（收集样本数据，有监督微调）：在人类标注的数据上微调出来的模型叫做 有监督的微调（supervised fine-tuning），这是训练出来的第一个模型



- 2) 训练奖励模型（收集排序数据，训练奖励模型）：
 - a) 给定一个问题，让上一步训练好的预训练模型 SFT 生成答案
 - b) GPT 每一次预测一个词的概率，可以根据这个概率采样出很多答案，通常来说可以用 beam search
 - c) 这里生成了四个答案，然后把这四个答案的好坏进行人工标注，进行排序标注
 - d) 有了这些排序之后，再训练一个奖励模型 (Reward Model, RM)，这个模型是说给定 prompt 得到输出，然后对这个输出生成一个分数，可以认为这个分数是一个奖励或者是打分，使得对答案的分数能够满足人工排序的关系（大小关系保持一致），一旦这个模型生成好之后，就能够对生成的答案进行打分